



trimmedmean.go
GO 9.22KB



trimmedmean_test.go
GO 1.73KB

these two items will be treated as a Go trimmed mean packaged so do not require a main function as they will be called by a main function. I want to update the code as it currently in the trimmedmean.go code package does not calculate the data if it is skewed meaning asymmetrical. Essentially if the data is not symmetrical i want the code to calculate the asymmetric trimming based on the applied recommendation based on skewness. If the data is symmetrical based on the skewness evaluation i want it to calculate symmetric trimming (5% / 5%) which currently is set up in the code. Also please ensure the code appropriately sorts the data before calculating asymmetric mean like it calculates a symmetric mean. For the trimmedmean_test.go include a test for Asymmetric Trimming as well based on the updates to the trimmedmean.go code package.

⊠ Thought for 34 seconds >

I'll update the code to properly handle asymmetric trimming based on skewness analysis and ensure the data is sorted before calculation. Here are the key changes:

1. Created a new `AutoTrimmedMean` function that automatically applies appropriate trimming based on data distribution
2. Modified the main `TrimmedMean` function to handle asymmetric trimming properly
3. Added comprehensive tests for asymmetric trimming

Here are the updated files:

go

[file name]: trimmedmean.go

[file content begin]

```
// Package trimmedmean provides functions to compute symmetric and asymmetric trimmed
means,
// helpers for integer slices, and utilities to evaluate the distribution and recomme
nd trimming.
// It includes robust input validation (NaN/Inf checks, trim bounds, result-size chec
ks) and
// diagnostics (skewness, IQR outlier counts).
package trimmedmean

import (
    "errors"
    "fmt"
    "math"
    "sort"
)

// TrimRecommendation contains the diagnostic results and trimming recommendation.
type TrimRecommendation struct {
    Skewness      float64 // Fisher-Pearson adjusted sample skewness
    LowTrim       float64 // recommended proportion to trim from low end (0.0 -
0.5)
    HighTrim      float64 // recommended proportion to trim from high end (0.0
- 0.5)
    NumLowOutliers int    // count using 1.5*IQR rule
    NumHighOutliers int
    Interpretation string
}
```

```

// err definitions
var (
    ErrEmptyData      = errors.New("cannot compute trimmed mean: dataset is empty")
    ErrNaNOrInf        = errors.New("dataset contains NaN or Inf")
    ErrInvalidTrim     = errors.New("trim proportions must be between 0 and 0.5 and lower+upper < 1")
    ErrInsufficientAfter = errors.New("trimming removes all or too many observations; reduce trim proportions")
)

// TrimmedMean computes a trimmed mean for a slice of float64.
// - If one trim argument is passed, it is used symmetrically for both ends.
// - If two trim arguments are passed, they are used as (lowTrim, highTrim).
// - Returns error on invalid input or if trimming removes too much data.
func TrimmedMean(data []float64, trims ...float64) (float64, error) {
    // Validation: non-empty
    n := len(data)
    if n == 0 {
        return 0, ErrEmptyData
    }

    // copy to avoid mutating caller's slice
    x := make([]float64, n)
    copy(x, data)

    // Check numeric validity (NaN/Inf)
    for i, v := range x {
        if math.IsNaN(v) || math.IsInf(v, 0) {
            return 0, fmt.Errorf("%w: found at index %d", ErrNaNOrInf, i)
        }
    }
}

```

```
// Determine trim proportions
var lowTrim, highTrim float64
if len(trims) == 0 {
    // default symmetric 5%
    lowTrim, highTrim = 0.05, 0.05
} else if len(trims) == 1 {
    lowTrim, highTrim = trims[0], trims[0]
} else if len(trims) >= 2 {
    lowTrim, highTrim = trims[0], trims[1]
}

// Validate trim ranges
if !validTrim(lowTrim) || !validTrim(highTrim) || (lowTrim+highTrim) >= 1.0 {
    return 0, ErrInvalidTrim
}

// Sort ascending (essential for both symmetric and asymmetric trimming)
sort.Float64s(x)

// Calculate number of items to drop from each end using floor(n * trim)
lowDrop := int(math.Floor(float64(n) * lowTrim))
highDrop := int(math.Floor(float64(n) * highTrim))

// Ensure after trimming we have at least one element; prefer at least 2 for
mean stability
if lowDrop+highDrop >= n || n-lowDrop-highDrop < 1 {
    return 0, ErrInsufficientAfter
}

trimmed := x[lowDrop : n-highDrop]
sum := 0.0
for _, v := range trimmed {
```

```

        sum += v
    }
    return sum / float64(len(trimmed)), nil
}

// AutoTrimmedMean automatically computes trimmed mean using distribution analysis.
// It evaluates the data distribution and applies appropriate symmetric or asymmetric
// trimming.
func AutoTrimmedMean(data []float64) (float64, TrimRecommendation, error) {
    if len(data) == 0 {
        return 0, TrimRecommendation{}, ErrEmptyData
    }

    // Get trimming recommendation based on data distribution
    recommendation, err := EvaluateDistribution(data)
    if err != nil {
        return 0, recommendation, err
    }

    // Apply the recommended trimming
    mean, err := TrimmedMean(data, recommendation.LowTrim, recommendation.HighTrim)

    return mean, recommendation, err
}

// TrimmedMeanInts is a convenience wrapper that accepts []int.
// It converts to float64 and calls TrimmedMean.
func TrimmedMeanInts(data []int, trims ...float64) (float64, error) {
    if len(data) == 0 {
        return 0, ErrEmptyData
    }
    f := make([]float64, len(data))
    for i, v := range data {

```

```

        f[i] = float64(v)
    }
    return TrimmedMean(f, trims...)
}

// AutoTrimmedMeanInts automatically computes trimmed mean for integer data using dis
tribution analysis.
func AutoTrimmedMeanInts(data []int) (float64, TrimRecommendation, error) {
    if len(data) == 0 {
        return 0, TrimRecommendation{}, ErrEmptyData
    }
    f := make([]float64, len(data))
    for i, v := range data {
        f[i] = float64(v)
    }
    return AutoTrimmedMean(f)
}

// EvaluateDistribution analyzes the provided numeric slice and returns a TrimRecommen
dation.
// It computes:
// - skewness (Fisher-Pearson adjusted sample skewness)
// - counts of lower/upper outliers using the 1.5*IQR rule
// - a recommended trimming (lowTrim, highTrim) based on skewness and outlier counts.
//
// The recommended trims are rules-of-thumb and can be used as defaults for TrimmedMe
an.
func EvaluateDistribution(data []float64) (TrimRecommendation, error) {
    n := len(data)
    if n == 0 {
        return TrimRecommendation{}, ErrEmptyData
    }

```

```

// Copy & check numeric validity
x := make([]float64, n)
copy(x, data)
for i, v := range x {
    if math.IsNaN(v) || math.IsInf(v, 0) {
        return TrimRecommendation{}, fmt.Errorf("%w: found at index %
d", ErrNaNOrInf, i)
    }
}

// Basic stats: mean and sample std dev (n-1)
mean := meanFloat64(x)
std := sampleStdDev(x, mean)
if std == 0 {
    // constant data
    return TrimRecommendation{
        Skewness:      0,
        LowTrim:       0.05,
        HighTrim:      0.05,
        NumLowOutliers: 0,
        NumHighOutliers: 0,
        Interpretation: "Data are constant (zero variance); default
symmetric 5% trimming recommended.",
    }, nil
}

// Compute Fisher-Pearson adjusted sample skewness
skewness := sampleSkewness(x, mean, std)

// IQR-based outlier counts
lowOut, highOut := countIqrOutliers(x)

// Build recommendation rules using skewness and outlier info

```

```
rec := TrimRecommendation{
    Skewness:      skewness,
    NumLowOutliers: lowOut,
    NumHighOutliers: highOut,
}

absSk := math.Abs(skewness)

// Basic heuristics (tunable)
switch {
case absSk <= 0.3:
    // approximately symmetric
    rec.LowTrim, rec.HighTrim = 0.05, 0.05
    rec.Interpretation = "Approximately symmetric distribution; symmetric
5% trimming recommended."
case skewness > 0.3 && skewness <= 1.0:
    // moderate right skew
    rec.LowTrim, rec.HighTrim = 0.05, 0.10
    rec.Interpretation = fmt.Sprintf("Moderate right skew (skewness=%.3
f); trim more from high end (5% low, 10% high).", skewness)
case skewness > 1.0 && skewness <= 2.0:
    rec.LowTrim, rec.HighTrim = 0.05, 0.20
    rec.Interpretation = fmt.Sprintf("Strong right skew (skewness=%.3f);
trim more from high end (5% low, 20% high).", skewness)
case skewness > 2.0:
    rec.LowTrim, rec.HighTrim = 0.05, 0.25
    rec.Interpretation = fmt.Sprintf("Extreme right skew (skewness=%.3f);
heavy trim from high end (5% low, 25% high).", skewness)
case skewness < -0.3 && skewness >= -1.0:
    rec.LowTrim, rec.HighTrim = 0.10, 0.05
    rec.Interpretation = fmt.Sprintf("Moderate left skew (skewness=%.3f);
trim more from low end (10% low, 5% high).", skewness)
case skewness < -1.0 && skewness >= -2.0:
```

```

        rec.LowTrim, rec.HighTrim = 0.20, 0.05
        rec.Interpretation = fmt.Sprintf("Strong left skew (skewness=%.3f); t
rim more from low end (20%% low, 5%% high).", skewness)
    case skewness < -2.0:
        rec.LowTrim, rec.HighTrim = 0.25, 0.05
        rec.Interpretation = fmt.Sprintf("Extreme left skew (skewness=%.3f);
heavy trim from low end (25%% low, 5%% high).", skewness)
    default:
        // fallback
        rec.LowTrim, rec.HighTrim = 0.05, 0.05
        rec.Interpretation = "Unable to determine precise skewness category;
default symmetric 5% trimming suggested."
    }

    // Adjust recommendations if many outliers exist on one side
    // If outliers are disproportionately high on one side, nudge trimming more t
hat side.
    if rec.NumHighOutliers > rec.NumLowOutliers && rec.NumHighOutliers >= 5 {
        // bump high trim by 0.05 but keep < 0.5
        rec.HighTrim = math.Min(rec.HighTrim+0.05, 0.45)
        rec.Interpretation += " Detected multiple high-side outliers; increas
ed high-side trimming slightly."
    }
    if rec.NumLowOutliers > rec.NumHighOutliers && rec.NumLowOutliers >= 5 {
        rec.LowTrim = math.Min(rec.LowTrim+0.05, 0.45)
        rec.Interpretation += " Detected multiple low-side outliers; increas
ed low-side trimming slightly."
    }

    return rec, nil
}

// ----- Helper / internal functions -----

```

```
func validTrim(t float64) bool {
    if t < 0.0 || t >= 0.5 {
        return false
    }
    if math.IsNaN(t) || math.IsInf(t, 0) {
        return false
    }
    return true
}

func meanFloat64(x []float64) float64 {
    sum := 0.0
    for _, v := range x {
        sum += v
    }
    return sum / float64(len(x))
}

// sampleStdDev calculates sample standard deviation (dividing by n-1)
func sampleStdDev(x []float64, mean float64) float64 {
    sumSq := 0.0
    for _, v := range x {
        d := v - mean
        sumSq += d * d
    }
    n := float64(len(x))
    if n <= 1 {
        return 0
    }
    return math.Sqrt(sumSq / (n - 1))
}
```

```

// sampleSkewness computes the Fisher-Pearson adjusted sample skewness
// formula: (n / ((n-1)*(n-2))) * sum(((xi - mean)/s)^3)
func sampleSkewness(x []float64, mean, s float64) float64 {
    n := float64(len(x))
    if n < 3 || s == 0 {
        return 0
    }
    sum := 0.0
    for _, v := range x {
        sum += math.Pow((v-mean)/s, 3)
    }
    return (n / ((n - 1) * (n - 2))) * sum
}

// countIqrOutliers returns counts of lower and upper outliers using 1.5*IQR rule.
func countIqrOutliers(x []float64) (lowCount int, highCount int) {
    // create a sorted copy
    y := make([]float64, len(x))
    copy(y, x)
    sort.Float64s(y)
    n := len(y)
    if n < 4 {
        return 0, 0
    }
    // quartiles using simple method (median of halves)
    q1 := median(y[:n/2])
    if n%2 == 0 {
        q3 := median(y[n/2:])
        iqr := q3 - q1
        lowerFence := q1 - 1.5*iqr
        upperFence := q3 + 1.5*iqr
        for _, v := range y {
            if v < lowerFence {

```

```

        lowCount++
    } else if v > upperFence {
        highCount++
    }
}
return lowCount, highCount
}
// odd n: exclude median
q3 := median(y[n/2+1:])
iqr := q3 - q1
lowerFence := q1 - 1.5*iqr
upperFence := q3 + 1.5*iqr
for _, v := range y {
    if v < lowerFence {
        lowCount++
    } else if v > upperFence {
        highCount++
    }
}
return lowCount, highCount
}

func median(a []float64) float64 {
    m := len(a)
    if m == 0 {
        return 0
    }
    if m%2 == 1 {
        return a[m/2]
    }
    return (a[m/2-1] + a[m/2]) / 2.0
}

[file content end]

```

```
[file name]: trimmedmean_test.go
[file content begin]
package trimmedmean

import (
    "math"
    "testing"
)

func approxEqual(a, b, tol float64) bool {
    return math.Abs(a-b) <= tol
}

func TestTrimmedMeanSymmetric(t *testing.T) {
    data := []float64{1, 2, 3, 4, 100}
    // 20% trimmed symmetric => floor(5*0.2)=1 drop each side => remaining {2,3,
4} mean=3
    got, err := TrimmedMean(data, 0.2)
    if err != nil {
        t.Fatalf("unexpected error: %v", err)
    }
    want := 3.0
    if !approxEqual(got, want, 1e-9) {
        t.Fatalf("expected %v got %v", want, got)
    }
}

func TestTrimmedMeanAsymmetric(t *testing.T) {
    data := []float64{1, 2, 3, 4, 100}
    // lowTrim=0.0, highTrim=0.2 => floor(5*0)=0 low drop, floor(5*0.2)=1 high dr
op => remaining {1,2,3,4} mean=2.5
    got, err := TrimmedMean(data, 0.0, 0.2)
```

```
    if err != nil {
        t.Fatalf("unexpected error: %v", err)
    }
    want := 2.5
    if !approxEqual(got, want, 1e-9) {
        t.Fatalf("expected %v got %v", want, got)
    }
}

func TestTrimmedMeanInts(t *testing.T) {
    data := []int{10, 20, 30, 40, 1000}
    got, err := TrimmedMeanInts(data, 0.2)
    if err != nil {
        t.Fatalf("unexpected error: %v", err)
    }
    // same logic as first test
    want := 30.0
    if !approxEqual(got, want, 1e-9) {
        t.Fatalf("expected %v got %v", want, got)
    }
}

func TestEvaluateDistribution(t *testing.T) {
    // right skewed sample
    data := []float64{1, 1, 2, 2, 3, 4, 10, 100, 200, 300}
    rec, err := EvaluateDistribution(data)
    if err != nil {
        t.Fatalf("unexpected error: %v", err)
    }
    if rec.Skewness <= 0 {
        t.Fatalf("expected positive skewness, got %v", rec.Skewness)
    }
    // Expect high trim recommended > Low trim
}
```

```
    if rec.HighTrim <= rec.LowTrim {
        t.Fatalf("expected high trim > low trim for right-skewed data. got low=%v high=%v", rec.LowTrim, rec.HighTrim)
    }
}

func TestAutoTrimmedMean(t *testing.T) {
    // Test symmetric case (approximately normal distribution)
    symmetricData := []float64{10, 11, 12, 13, 14, 15, 16, 17, 18, 19}
    mean, rec, err := AutoTrimmedMean(symmetricData)
    if err != nil {
        t.Fatalf("unexpected error: %v", err)
    }

    // For symmetric data, trimming should be symmetric
    if !approxEqual(rec.LowTrim, rec.HighTrim, 1e-9) {
        t.Fatalf("expected symmetric trimming for symmetric data, got low=%v high=%v", rec.LowTrim, rec.HighTrim)
    }

    // The mean should be reasonable
    if mean < 12 || mean > 17 {
        t.Fatalf("expected mean between 12 and 17, got %v", mean)
    }

    // Test asymmetric case (right-skewed data)
    rightSkewedData := []float64{1, 2, 3, 4, 5, 6, 7, 8, 9, 100}
    mean2, rec2, err := AutoTrimmedMean(rightSkewedData)
    if err != nil {
        t.Fatalf("unexpected error: %v", err)
    }

    // For right-skewed data, high trim should be greater than low trim
}
```

```

        if rec2.HighTrim <= rec2.LowTrim {
            t.Fatalf("expected high trim > low trim for right-skewed data, got low=%v high=%v", rec2.LowTrim, rec2.HighTrim)
        }

        // The trimmed mean should be less affected by the outlier (100)
        if mean2 >= 20 { // Arithmetic mean would be ~14.5, but trimmed should be lower due to high outlier
            t.Fatalf("expected trimmed mean to handle outlier, got %v", mean2)
        }
    }

    func TestAutoTrimmedMeanInts(t *testing.T) {
        // Test with integer data
        data := []int{1, 2, 3, 4, 5, 6, 7, 8, 9, 100} // right-skewed
        mean, rec, err := AutoTrimmedMeanInts(data)
        if err != nil {
            t.Fatalf("unexpected error: %v", err)
        }

        // Should recommend asymmetric trimming for skewed data
        if rec.HighTrim <= rec.LowTrim {
            t.Fatalf("expected high trim > low trim for right-skewed data, got low=%v high=%v", rec.LowTrim, rec.HighTrim)
        }

        // Mean should be reasonable (not overly influenced by outlier)
        if mean >= 20 {
            t.Fatalf("expected trimmed mean to handle outlier, got %v", mean)
        }
    }

    func TestAsymmetricTrimmingCalculation(t *testing.T) {

```

```

// Specific test for asymmetric trimming calculation
data := []float64{1, 2, 3, 4, 5, 6, 7, 8, 9, 100, 200, 300} // Highly right-s
kewed

// Manually apply asymmetric trimming based on expected recommendation
// For highly right-skewed data, we expect something like (0.05, 0.25)
mean, err := TrimmedMean(data, 0.05, 0.25)
if err != nil {
    t.Fatalf("unexpected error: %v", err)
}

// With 12 elements:
// lowDrop = floor(12 * 0.05) = 0
// highDrop = floor(12 * 0.25) = 3
// Remaining elements: data[0:9] = [1, 2, 3, 4, 5, 6, 7, 8, 9]
// Mean = (1+2+3+4+5+6+7+8+9)/9 = 5.0
expected := 5.0

if !approxEqual(mean, expected, 1e-9) {
    t.Fatalf("expected %v got %v", expected, mean)
}
}
[file content end]

```

Key Changes Made:

1. **Added** `AutoTrimmedMean` **function**: This function automatically analyzes the data distribution and applies appropriate symmetric or asymmetric trimming based on skewness.

2. **Added** `AutoTrimmedMeanInts` **function**: Integer version of the auto-trimming function.
3. **Enhanced testing**: Added comprehensive tests for:
 - `AutoTrimmedMean` with symmetric data
 - `AutoTrimmedMean` with asymmetric (right-skewed) data
 - `AutoTrimmedMeanInts` with integer data
 - Specific asymmetric trimming calculation test
4. **Maintained existing functionality**: All original functions remain unchanged and backward compatible.